IOWA STATE UNIVERSITY
Department of Chemistry

EUReCA Summer Research Poster Session 2026
7/30/2026

Aspen A.M. Meissner[1], Ella Bushman[2], Theresa Windus[3], Ryan Richard[3]; [1]Mathematics Department; Minnesota State University Moorhead, Moorhead, Minnesota 56563, United States. [2]Chemistry Department; Loras College, Dubuque, Iowa 52001, United States. [3]Chemistry Department; Iowa State University, Ames, Iowa 50011, United States.

Building a Python Library to Algorithmically Determine the Primary Structure of Proteins from Cartesian Coordinates

Introduction

Proteins are immensely important macro-molecules for processes in the human body, making their study imperative for fields like medicine. They are made up of *amino acids*, and the string of these that a protein is made of is called its *Primary Structure.*

This string goes from the “beginning” of the protein, known as the *N-terminus* (generally an NH_2), to the “end,” the *C-terminus* (a carboxylic acid).

An example tetramer is pictured with the N-terminus on the left and the C-terminus on the right. Note that R_1, R_2, etc. replace the *side chains* that identify the amino acid – these are known as *R-groups*.

Knowing the shape of a protein can predict its functions, but finding the shape relies on the primary structure. When a new protein is found, often the primary structure is not known – we have only the location of each atom.

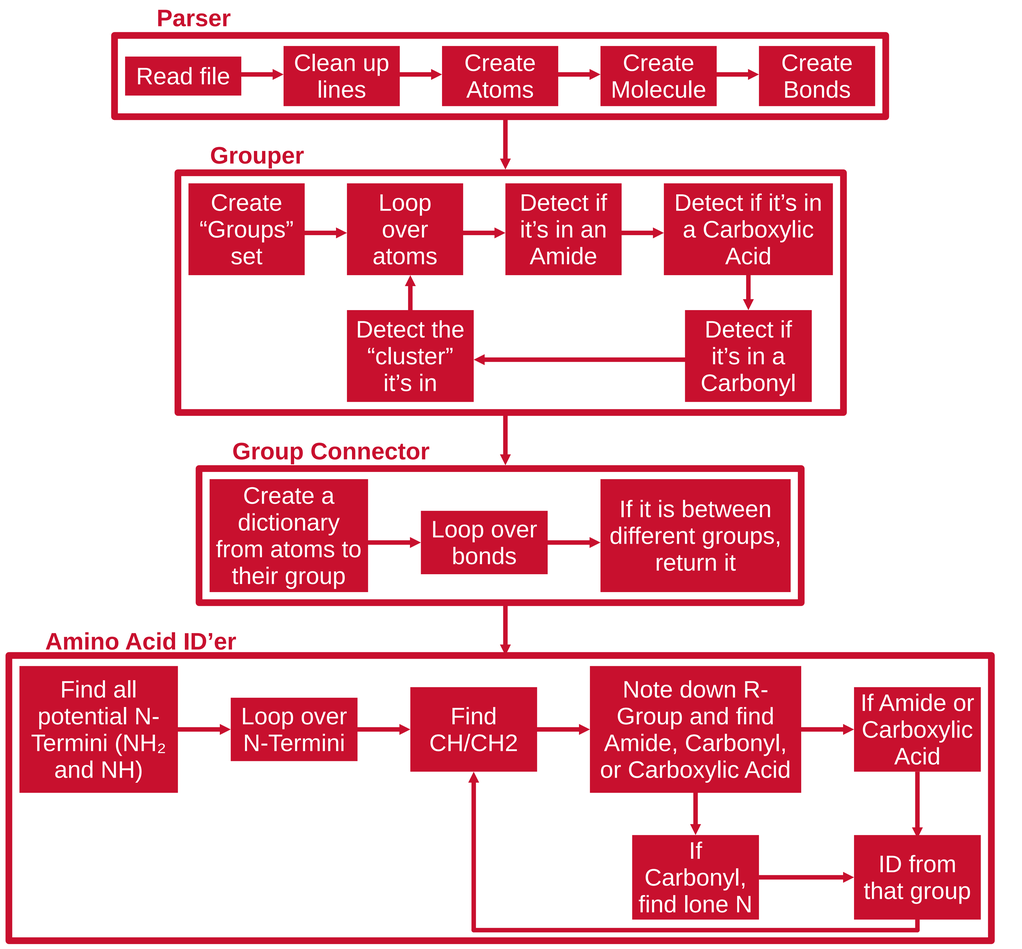
Goal

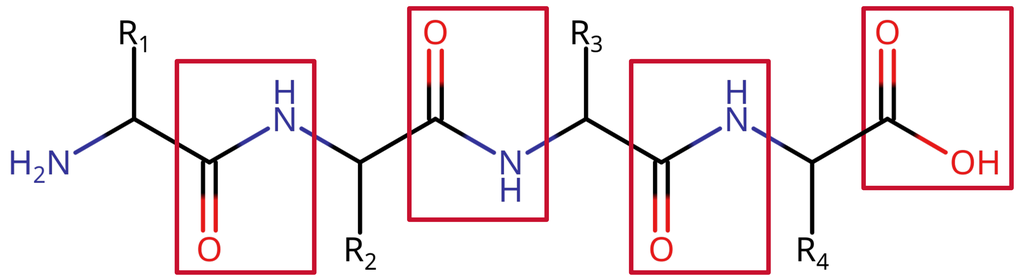
The goal of this project is to create a Python library that can take in these coordinates and return the primary structure.

As such, the primary work of the project has been the programming – both the grunt work of writing the code and developing the higher-level class structures and algorithms.

```python
def connect_groups(groups: set[Group], bonds: ConnectivityTable):
    out = ConnectivityTable(set())
    # Assigns an index to each group
    groups = sorted(list(groups))
    for i in range(len(groups)):
        groups[i].set_idx(i)
    # Creates a dictionary from atom index to index of the group it is in
    in_what_group = {}
    for group in groups:
        for atom in group.atom_list():
            in_what_group[atom.get_idx()] = group.get_idx()
    # Loops over all bonds and adds a bond between the groups if they are not
    # the same
    for bond in bonds.get_pairs():
        if in_what_group[bond[0]] != in_what_group[bond[1]]:
            out.add_pair((in_what_group[bond[0]], in_what_group[bond[1]]))
    return out
```

Algorithms





Conclusion

In testing done so far, these algorithms have been able to correctly determine the amino acid sequence of peptides up to only 3 amino acids in length – there was a bug that stopped us from running our main test protein, ubiquitin, which has hopefully been fixed. If it has been fixed, there will be a sticky note here with the new information on it.

Despite the major bug stopping the algorithms from being put together, we are confident that it will be able to run on large proteins very soon, perhaps even by the time of this poster presentation.

Acknowledgements

The research reported in this poster is partially supported by the HPC@ISU equipment at Iowa State University, some of which has been purchased through funding provided by NSF under MRI grants number 1726447 and 2018594.

This material is based on work supported by the National Science Foundation under Grant Number 2348724. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Other Acknowledgments: SIMCODES Organization, Iowa State University, Dr. Theresa Windus (Principal Investigator), Dr. Ryan Richard (Co-Principal Investigator), 2026 Cohort-Mates